

Niveau : II1 Date : 21/03/2025 Durée : 2 heures Nombre de pages : 5	Documents : Non autorisés Calculatrice : Autorisée Annexe : Non Barème à titre indicatif (4+6+10)	 Ecole Nationale des Sciences de l'Informatique Université de la Manouba A.U. 2024/2025
Enseignantes : I. AMDOUNI, I. BOUKHRIS, R. CHEBIL, I. FLISS, A. HEDHILI.	Réf : ID.104 Version : 01	
DEVOIR SURVEILLÉ Programmation Orientée Objet		

Exercice 1 (4pts : 1 pt *4)

1) Considérons le code suivant:

```
#include <iostream>
#include <string>
using namespace std;
class Robot
{
public:
    Robot() : energie_(100) {};
    int getEnergie() const { return energie_; }
    void setEnergie(int energie) { energie_ = energie; }
    void travailler()
    {
        energie_ -= 10;
        cout << "Énergie restante : " << energie_ << endl;
    }
    bool operator==(Robot r) { return (nom == r.nom); }
private:
    int energie_;
    string nom;
};

Robot chercher(Robot*& usine, Robot r, int pos)
{
    cout << "Recherche d'un robot" << endl;
    if (usine[pos] == r) return r; }

void recharger(Robot robot)
{
    robot.setEnergie(robot.getEnergie() + 20);
    cout << "Énergie après recharge : " << robot.getEnergie() << endl;
}

void reparer(Robot robot)
{
    cout << "Énergie avant réparation : " << robot.getEnergie() << endl;
    robot.setEnergie(100);
    cout << "Énergie après réparation : " << robot.getEnergie() << endl;
}

int main()
{
    Robot r1;
    Robot* usine = new Robot[3];
    chercher(usine, r1, 1);
    r1.travailler();
    recharger(r1);
    recharger(r1);
    reparer(r1);
    return 0;}
```

Combien d'objets de type Robot sont créés lors de l'exécution de ce programme ?

- a) Un
- b) Trois
- c) Quatre
- d) Cinq
- e) Huit

2) Soit le programme suivant :

```
#include <iostream>
using namespace std;
class Base {
public:
    Base() { cout << "Base "; }
    ~Base() { cout << "~Base "; }
};
class Derivee1 : public Base {
public:
    Derivee1() { cout << "Derivee1 "; }
    ~Derivee1() { cout << "~Derivee1 "; }
};
class Derivee2 : public Derivee1 {
public:
    Derivee2() { cout << "Derivee2 "; }
    ~Derivee2() { cout << "~Derivee2 "; }
};
int main() {
    Derivee2 d;
}
```

Quel sera l'affichage du programme ?

- a) Base Derivee1 Derivee2 ~Derivee2 ~Derivee1 ~Base
- b) Base Derivee1 Derivee2 ~Base ~Derivee1 ~Derivee2
- c) Derivee2 Derivee1 Base ~Base ~Derivee1 ~Derivee2
- d) Derivee2 Derivee1 Base ~Derivee2 ~Derivee1 ~Base
- e) Ne compile pas

3) Soit le code suivant:

<pre>class G { int a; public: G(int val) : a(val) {} void setA(int val) { a = val; } int getA() const { return a; } }; class H { int b; public: H(int val) : b(val) {} inline void setB(int val); int getB() const { return b; } }; inline void H::setB(int val) { this->b = val;}</pre>	<pre>int main() { G objG(5); H objH(10); objG.setA(15); const G& refG = objG; refG.getA(); objH.setB(20); const H& refH = objH; refH.getB(); return 0; }</pre>
---	--

Quelle(s) affirmation(s) est (sont) correcte(s):

- a) La méthode setA dans la classe G est plus rapide que setB dans la classe H car cette dernière est inline.
- b) Le programme est correct et les appels de méthode sur des objets const dans main sont valides.
- c) La méthode getA dans la classe G ne peut pas être appelée sur une référence const car elle n'est pas marquée const.

4) Au niveau d'une classe, peut-on surcharger le constructeur et le destructeur. Expliquez votre réponse ?

Exercice 2 (6 pts)

On considère, dans cet exercice, les classes suivantes :

```
#include <iostream>
#include <string>
using namespace std;

class Document {
private:
    string titre;
public :
    Document(string t = "") : titre(t) {
        cout << "Document créé : " << titre << endl ; }
    ~Document() {
        cout << "Document supprimé : " << titre << endl; }
    string getTitre() { return titre ; }
};

class Imprimante {
private:
    Document* tete;
    Document* queue;
public :
    Imprimante() {
        tete = new Document;
        queue = new Document;
        cout << "Imprimante initialisée"<< endl;
    }
    Imprimante(string titreTete, string titreQueue) {
        tete = new Document(titreTete);
        queue = new Document(titreQueue);
        cout << "Imprimante initialisée avec deux documents" << endl;
    }
    ~Imprimante() {
        delete tete;
        delete queue;
        cout << "Imprimante supprimée" << endl;
    }
    void afficher() {
        cout << "Tête : "<<tete->getTitre()<< " | Queue : "<<queue->getTitre()<<endl;    }
};

void testerCopie(Imprimante imp) {
    imp.afficher();
}
```

- 1) On considère les instructions suivantes. Pour chacune d'elles, indiquer les lignes affichées, si l'instruction génère un problème, et si oui décrire ce problème en remplissant le tableau suivant :

Numéro instruction	Affichage	Si pas de problème écrire NON, sinon décrire ce problème	Citer la solution
/*1*/			
...			
/*7*/ (fin du bloc)			

```
int main() {
    /*1*/Imprimante imp1 ("rapport PFE", "attestation reussite");
    /*2*/Imprimante imp2 ;
    /*3*/Imprimante imp3 ("devis PC", "facture PC");
    /*4*/testerCopie(imp1);
    /*5*/imp2 = imp3;
    /*6*/imp2.afficher();
    /*7*/ }
```

- 2) Implémenter les solutions citées dans la première question en indiquant à quelle classe elles appartiennent.

Problème (10 points)

On s'intéresse dans ce problème à une classe 'Polynome' représentant des polynômes à une variable et à coefficients entiers ayant la forme suivante :

$A = a_0x^0 + a_1x^1 + a_2x^2 + \dots + a_Nx^N$ avec $N \geq 0$ représentant le degré du polynôme et a_i un entier naturel représentant le coefficient du degré i du polynôme.

On déclare cette classe comme suit :

```
class Polynome
{
private:
    int degre ;           //degré du polynome
    int* Tab ;           //tableau dynamique de coefficients
/*Par exemple le tableau:{1,2,0,3} représente le polynôme: 1x^0+2x^1+0x^2+3x^3 */
public:
    Polynome(int d);
/*permet de créer un polynôme de degré d à coefficients nuls */

    Polynome(int* A, int d);
/*permet de créer un polynôme de degré d dont les coefficients sont initialisés
par les éléments du tableau A*/

    void set_coef(int d, int coef);
/*permet de remplacer par coef le coefficient de degré d s'il existe */
...} ;
```

Pour gagner de l'espace et du temps, donner directement les implémentations des méthodes demandées dans chaque question. Dans le cas où la fonction est amie à la classe, écrire le mot clé devant son entête. On ne demande d'implémenter que les setters et/ou getters dont vous avez besoin dans le problème ou qui sont demandés explicitement.

Questions

1. Donner l'implémentation des méthodes déclarées dans la classe `Polynome` ainsi que toute autre méthode nécessaire à son bon fonctionnement. (3.5 pts)
2. On souhaite effectuer les manipulations suivantes sur les Polynômes :
Soient $A = 5x^0 + 3x^1 + 1x^2$ et $B = 0x^0 + 2x^1 + 4x^2$
 $++A = 6x^0 + 4x^1 + 2x^2$
 $A*B = 0x^0 + 10x^1 + 26x^2 + 14x^3 + 4x^4$
 $A[2] = 15$ (Evaluation du polynôme A pour un entier $x=2$)

Donner les surcharges d'opérateurs nécessaires permettant d'assurer ces manipulations. (2.5 pts)

3. Ajoutez les méthodes nécessaires permettant d'exécuter le programme suivant : (2pts)

```
int main() {  
    int T[] = {1, 2, 0, 3} ;  
    Polynome p1(T, 3), p2(2);  
    cout << p1 ;           // affiche : 1x^0+2x^1+0x^2+3x^3  
    p2 = p1++ ;  
    cout << p2 ;           // affiche : 1x^0+2x^1+0x^2+3x^3  
    cout << p1 ;           // affiche : 2x^0+3x^1+1x^2+4x^3  
    return 0 ;  
}
```

4. On se propose maintenant de généraliser la classe `Polynome` afin qu'on puisse l'utiliser non seulement pour des coefficients de type entier mais aussi de types flottant, double ou short.... Donner **uniquement la déclaration** de la classe '`Polynome_gen`' permettant d'assurer cette généricité. (1pt)
5. Ecrire une fonction main permettant de tester la classe précédemment déclarée en créant un polynôme p1 avec des coefficients nuls de type short et un polynôme p2 avec des coefficients non nuls de type double et d'afficher les deux objets créés. (1pt)

Bon travail